

测试与 质量保障

单元测试 · 端到端验证 · 持续集成

测试被视为 Ergalics Studio 代码库的一等公民：核心逻辑在 Node 环境用 Vitest 做单元测试，产品行为用 Playwright-core 驱动无头浏览器对生产预览做端到端验证，GitHub Actions 持续集成把两者串成回归网。本篇给出完整的测试地图：每个测试组覆盖什么、端到端套件验证什么、构建门禁如何设卡，以及贯穿全库的五条质量原则。

当前质量基线

417

单元测试
46 个文件

10

端到端套件
真实浏览器

2e-6

GPU 与 CPU
数值一致性

0

端到端断言的
控制台错误容忍

目 录

一、单元测试 (Vitest)

1.1 运行方式与目录组织

1.2 测试组覆盖地图

1.3 值得注意的测试技术

二、端到端测试 (Playwright-core)

三、持续集成与构建门禁

四、质量原则

03

03

04

06

06

08

08

阅读提示：本篇与前面各篇——呼应——第二篇说"核心服务可在 Node 环境测试"， 第五篇说"GPU 与 CPU 数值一致"，都可以在本篇找到对应的测试组与端到端套件。

一 单元测试 (Vitest)

1.1 运行方式与目录组织

运行方式：npm test 执行全部单测；npm run verify 先做类型检查（tsc 严格模式，零发布）再跑单测，是贡献指引中要求保持绿色的组合检查。

测试文件按子系统组织在 tests/ 目录下：blocks（9 个文件，区块系统）、core（2 个，计算服务与 WGSL）、editor（8 个，编辑器与 IR）、io（2 个）、plot（1 个）、repro（1 个）、stats（6 个），其余 17 个在根目录覆盖插件运行时、沙箱与各插件族。

46

测试文件

417

单元用例

29

子系统目录文件

17

根目录插件与沙箱

目录组织与源码组织严格对应：改了哪个子系统，就跑哪个目录的测试；贡献者从测试目录即可反推系统的模块划分。测试环境的运行环境为 node，不需要浏览器——这依赖于全库“纯函数优先、副作用收口”的架构纪律（见第四篇与第七篇的质量原则）。

1.2 测试组覆盖地图（一）

测试组	覆盖内容
fileFormat	魔数与扩展名检测、格式匹配
cspkg	ZIP 解析、清单校验、可信入口执行
sandbox	RPC 编解码、遗留回退、经 FakeWorker 驱动真实运行时的端到端往返
i18n	语言切换、回退、沙箱相关文案键
appStore	横幅、通知、性能告警
wasm	WASM 加载器重试策略（三次、间隔一秒），模块级 mock
gpuService	计算服务设备门控与 CPU 回退
gpuCompute	WGSL 模板生成、缓冲打包、CPU 积分器、粒子回退一致性
core/wgsl	直方图、热力图、点云 GPU 内核：模板输出、参数打包、输出尺寸、CPU 一致性

这一组测试对应第二、三、五篇的架构承诺：文件路由真的按魔数识别、沙箱 RPC 真的能往返、每个 GPU 内核真的有数学一致的 CPU 孪生。架构文档里的每一句"保证"，在这里都有一个具体的测试组背书。

1.2 测试组覆盖地图（二）

测试组	覆盖内容
builtinPlugins	等值线网格归一化、散点解析、N-Body、蛋白质、粒子、图像查看器等插件逻辑
sciencePlugins	三维 N-Body 内核与动量守恒、蛋白质力导向布局与连通分量、无数据守卫
geoPhysicsPlugins	流体 WGSL 内核与 CPU 步进、波动方程、双摆物理、GeoJSON 解析
structure-sim	结构力学求解：杆件轴力、利用率、超载垮塌、空态守卫、重置语义
pluginBugfixes	已修复插件缺陷的回归用例（如插件进入即暂停、Run 数据守卫）
dataPlugins	误差带行、矩形树层级、QQ 概率变换等解析辅助
blocks 系列	数据表运算、目录、编译器、执行器、几何、统计区块、文件数据、操作符、可视化桥接
editor 系列	共享 IR 校验与往返、Blockly JSON 与 IR 互转、解释器、JS 与 Python 代码生成、三模式 sync-threeway、积木 i18n、编辑器状态库、Studio API、代码示例
pluginRuntime	插件生命周期、市场门控、加载失败重试
repro、io、plot、stats 系列	可复现性内核、NetCDF 与辅助工具、绘图引擎、统计内核六个模块

几个值得展开的断言：N-Body 测试校验动量守恒，是物理正确性级别的验证而不仅是"不报错"；结构力学的重置语义测试保证"重置只重放已加载数据"这条产品承诺不被回归；三模式 sync-threeway 测试为积木、流程、代码的双向互转兜底，是共享 IR 架构的安全网。

1.3 值得注意的测试技术

- **模块级缓存重置**：模块级缓存（如 WASM 加载器）在每个用例前用 resetModules 加 doMock 重置，再动态导入新实例。
- **FakeWorker 驱动真实运行时**：Worker RPC 用 FakeWorker 驱动真实的插件 Worker 运行时做端到端往返，无需浏览器。
- **严格模式兼容**：严格模式下 new Function 无法同时使用 use strict 指令与默认参数值，遗留沙箱为此使用普通参数并显式传 undefined。

端到端测试 (Playwright-core)

每套脚本各自启动 vite preview 生产预览，驱动无头 Edge（默认取系统安装路径），断言真实像素与零控制台错误：

套件	端口	覆盖内容
smoke-test	4173	启动、自动加载插件、响应式参数、项目恢复
verify-ui	4173	布局尺寸、主题切换、画布可见性、插件列表
verify-fixes	4177	所有示例插件正确渲染其示例数据
verify-3d	4199	宿主 Three.js 场景中的三维点云
verify-plugins	4198	三维与二维表面可见性互斥、等值线、散点、龙卷风示例

二、端到端测试 (Playwright-core) (续)

套件	端口	覆盖内容
verify-webgpu	4289	无头 Edge (SwiftShader) 中的真实 WebGPU: GPU 与 CPU 数值一致性 (约 2e-6 以内)、粒子插件 GPU 提示断言
verify-block-mode	4173	积木编辑器: 模式切换、编译、运行、积木到代码同步
verify-code-mode	4175	Monaco 加 Pyodide: 运行 Python 程序、控制台、变量面板、绘图画布
verify-ai-samples	4173	AI 训练: 加载全部四个样本
verify-ai-training	4173	AI 训练器: 激活、TF.js 训练、损失曲线、模型切换重置、决策边界、MNIST CNN 网格

截图写入临时目录供人工目检。贡献指引要求新插件或新功能附带端到端检查。

端到端套件的分工刻意避免重叠: smoke 与 verify-ui 管应用壳, verify-fixes 管每个插件的示例渲染, verify-3d 与 verify-plugins 管视口互斥, verify-webgpu 管计算正确性, block-mode、code-mode 与两个 AI 套件管三种"重型"能力。每个套件只回答一个问题, 失败时能直接定位到能力域。

三 持续集成与构建门禁

GitHub Actions workflow 覆盖三条线：单元测试、端到端测试与 GitHub Pages 部署；CI 环境中的 WASM 模块由存根生成脚本提供，避免在云端安装完整 Rust 工具链。

生产构建命令链自带门禁：WASM 编译、TypeScript 类型检查、Vite 构建、文档站点拷贝，任一步失败即中止。类型检查（tsc 严格模式，noEmit）同时充当 lint，保持零 any 级别的约束。

四 质量原则

1. **纯函数优先**：编译器、统计内核、绘图引擎、代码生成与 IR 转换全部是纯函数，可在 Node 环境直接测试。
2. **副作用收口**：区块系统里只有渲染桥接一个带副作用的模块；执行器异步但无副作用，保证可测性。
3. **数值一致性兜底**：每个 GPU 内核都有数学一致的 CPU 实现，单测与端到端两层校验二者一致。
4. **边界显式化**：边界输入抛结构化错误、空数据启动有守卫提示、环境缺失如实自检报告，杜绝静默失败与静默降级。
5. **回归防御**：已修复的插件缺陷沉淀为 pluginBugfixes 回归用例，避免复发；i18n 键值在测试中校验齐备。

五条原则合起来是同一句话：让正确性可以被机器验证。凡是能写成纯函数的就不碰 DOM，凡是修过的 bug 就留下测试，凡是数学承诺就有数值断言。测试不是开发之后的补救，而是架构本身的选择。